



UNIVERSIDADE FEDERAL DE ALAGOAS – CAMPUS ARAPIRACA
BACHAREL EM CIÊNCIA DA COMPUTAÇÃO

ADRYAN VICTOR OLIVEIRA GOMES
ARTUR BARBOSA SILVA
ELIAS SALES DE MAGALHÃES
SAULO FIRMO DO NACIMENTO

OTECA

Relatório de Projeto de Sistema de Biblioteca

Arapiraca - AL

2026

ADRYAN VICTOR OLIVEIRA GOMES

ARTUR BARBOSA SILVA

ELIAS SALES DE MAGALHÃES

SAULO FIRMO DO NACIMENTO

OTECA

Relatório de Projeto de Sistema de Biblioteca

Relatório apresentado à Universidade Federal
de Alagoas como projeto da disciplina de
Estrutura de Dados como requisito para AB2.

Professor: Patrick Brito

Arapiraca – AL

2026

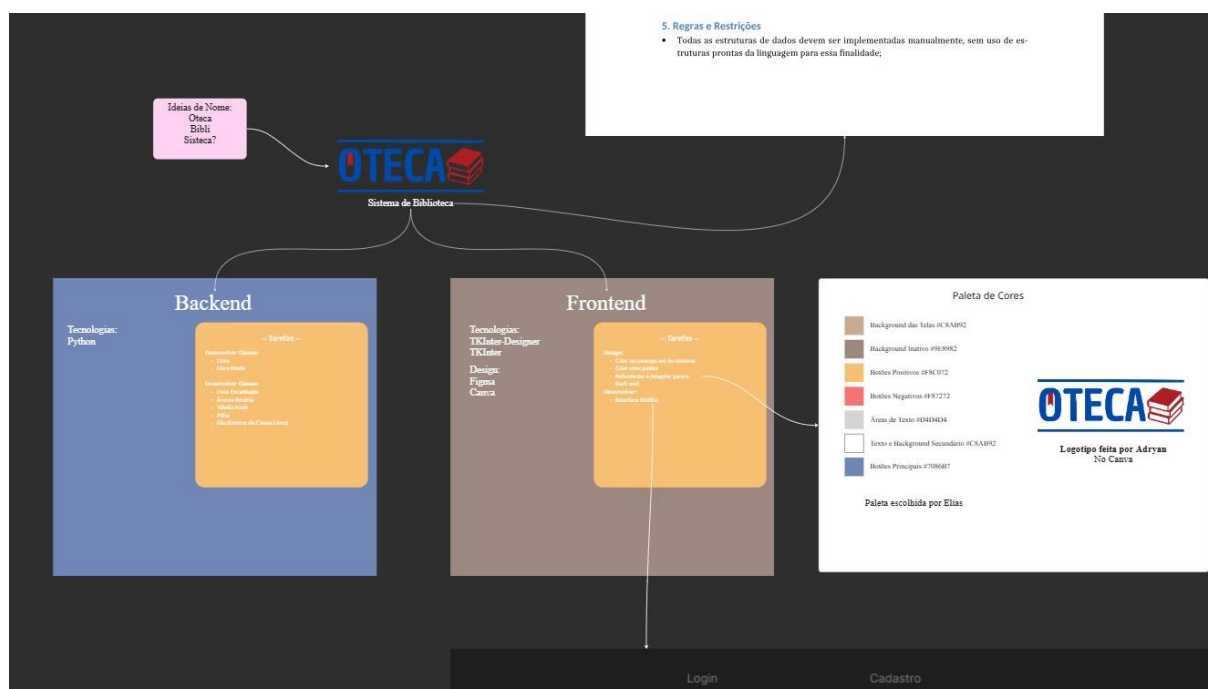
SUMÁRIO

1.	Análise de Documentação e Processo Criativo	4
2.	Backend - Sistema de Biblioteca	5
2.1.	Tabela Hash.....	5
2.2.	Fila	6
2.3.	Pilha	6
2.4.	Lista Encadeada.....	7
2.5.	Árvore Binária de Busca (BST)	7
2.6.	Biblioteca	7
3.	Front-End - OTECA.....	9

1. ANÁLISE DE DOCUMENTAÇÃO E PROCESSO CRIATIVO

Primeiramente foi realizado a leitura da documentação para criar a base criativa do projeto, buscando a partir das informações desenvolver seguindo as restrições e pedidos feitos pelo professor orientador.

No primeiro dia que foi anunciado o Projeto foi criado um repositório no GitHub, cujo link é <https://github.com/zfirmo/Projeto-ED-Sistema-de-Biblioteca>, e também foi criado uma pasta no Drive para todos terem acesso aos documentos que não vão para o repositório devido ao arquivo `.gitignore`, e foi criado um *board* no site Miro para fazermos o fluxo de ideias das tecnologias que utilizaremos.



Mapa Mental com ideias e tarefas para cada membro da equipe.

A utilização e meio de como utilizar está no arquivo `README.md` do repositório, porém vou esclarecer aqui, como foi exigido na documentação que o código tem que ter uma maneira de rodar apenas no terminal é só executar o arquivo `main.py`, para ir para a aplicação com interface gráfica basta executar o arquivo `oteka.py`.

O processo de começo parecia simples e até que no começo foi, porém quando foi amontoando as funções e cada uma tinha integração com outra foi criando uma dificuldade a mais, interessante para o projeto, nas outras sessões do relatório vai ter a descrição completa da funcionalidade do código, desde já obrigado professor pela oportunidade.

2. BACKEND - SISTEMA DE BIBLIOTECA

Seguindo a documentação desenvolvemos cada classe se atentando aos requisitos, primeiro foi criado a classe `Livro` para receber os atributos tipo `string` (`self`, ISBN, Título, Autor) e tipo `int` (ano de publicação, quantidade de exemplares), com essa classe definida para alocar os livros nas estrutura de dados necessárias criamos a classe `Livro_Node` que recebe (`self`, `livro`), e o importante que é os atributos `self.proximo`, para a lista encadeada, `self.direita` e `self.esquerda`, para a Árvore Binária e `qtd_máxima_original` guarda a quantidade de exemplares no cadastro do livro, evitando que possam devolver um livro que já existe que está com todos os exemplares disponíveis, e usado também para criar uma fila de espera individual por livro para os usuários.

Após temos a parte de usuários utilizadas nas funcionalidades de empréstimo e fila de espera, cada usuário tem um nome e um id, a classe `Usuario_Node` é usada para a fila de espera, ele recebe dois parâmetros (`self`, `usuario`) e os atributos usuário e próximo que aponta para o próximo usuário na lista de um livro, os demais *nodes* vão ser abordados em suas respectivas classes que utilizam os nodes, por exemplo o `HashNode` e o `Vetor_Estatico` serão abordados na parte do relatório da classe `Tabela_Hash`.

2.1. Tabela Hash

Primeiro começamos pela classe de `Vetor_Estatico` para montar a base usada na `Tabela_Hash`, a classe recebe (`self`, `tamanho`), para definir um tamanho fixo para nossa “Lista” e colocar funções que servem como ponteiros funcionando como se fosse uma lista, com os métodos `set` e `get`, `set` servindo para alocar em um índice específico e o `get` para pegar o valor de um índice específico.

Na classe `Tabela_Hash` ela é iniciada com os parâmetros (`self`, `tamanho=50`), e que inicia um vetor estático alocando 50 slots de memória para a Biblioteca, o método `_funcao_hash` pega o ISBN que é uma string em um número de índice (0 a 49) ele soma os dígitos e usa o operador de resto da divisão o `%` para garantir que o resultado caiba no vetor estático.

Os métodos de inserção, busca e remoção, sempre calculam o índice primeiro, em inserção ele cria um `Hash_Node`, uma classe que recebe os atributos `self.livro` que recebe o livro e `self.proximo` que guarda a posição do próximo elemento, se não houver nenhum

Hash_Node no vetor (`self.baldes.get(posição) is None`) ele usa o método da classe de Vetor_Estatico `set` e aloca o `novo_node` na sua posição, se houver ele pega a posição do e enquanto isso ele entra em um *looping* se ele achar o ISBN ele sai do *loop*, se ele chegar ao final do vetor (`atual.proximo is None`) ele termina o *loop*, após isso ele determina o atual como o próximo do atual. No método de busca ele vai até o índice e percorre a lista encadeada (usando `atual.proximo`) até achar o ISBN correto, no método de remoção a função é parecida à busca, mas com um controle extra (`anterior`). Se o livro for encontrado, o `anterior` aponta para o próximo do atual, "pulando" o nó que será removido e desconectando-o da lista.

2.2. Fila

A classe Fila foi implementada para gerenciar os usuários em espera de forma organizada, seguindo o princípio *FIFO (First In, First Out)*, onde o primeiro usuário a entrar na espera é o primeiro a ser atendido quando um livro é devolvido. A classe utiliza dois ponteiros, início e fim. O início aponta para o próximo usuário da fila a ser atendido, enquanto o fim aponta para a última pessoa adicionada.

A classe recebe os métodos de enfileirar, desenfileirar e remoção específica, no método de enfileirar ele recebe um objeto `Usuario`, cria um `Usuario_Node` e o posiciona no final da fila. Se a fila estiver vazia, esse nó torna-se tanto o início quanto o fim da estrutura. No método `desenfileirar` remove o nó do início, atualizando o ponteiro para o próximo da lista. Se a fila ficar vazia, o ponteiro `fim` é resetado para `None`, no método `remover_usuario_da_fila` que foi implementado para dar suporte à funcionalidade de desfazer. Ele percorre a fila buscando um nome de usuário específico e ajusta os ponteiros `proximo` dos nós adjacentes para "desconectar" o usuário da fila, independentemente de sua posição original.

2.3. Pilha

A Pilha é a estrutura que garante a rastreabilidade das operações, utilizando o princípio *LIFO (Last In, First Out)*. A pilha armazena objetos do tipo `Op_Node`, que contêm o tipo da operação realizada (`EMPRESTIMO`, `DEVOLUCAO` ou `ENTROU_FILA`), o ISBN do livro e, opcionalmente, o nome do usuário. Ela recebe os métodos de empilhar e desempilhar, no método `empilhar` ele insere novos registros no topo da pilha, fazendo com que o novo elemento aponte

para o antigo topo. Já no método `desempilhar` remove-se o registro atual, permitindo que o sistema retorne ao estado anterior da última transação efetuada.

2.4. Lista Encadeada

A `Lista_Encadeada` serve como um registro linear e dinâmico de todos os livros cadastrados no sistema, utilizando o `Livro_Node` para armazenar o livro e um ponteiro próximo. A cada novo cadastro, a lista percorre seus nós até encontrar o último elemento (`None`) e insere o novo livro ali. No método de remoção o método `remover_por_isbn` percorre a lista mantendo uma referência ao nó anterior. Ao encontrar o ISBN desejado, o anterior é conectado diretamente ao sucessor do nó atual, removendo o livro da lista.

2.5. Árvore Binária de Busca (BST)

Na classe de `Arvore_Binaria` ela é responsável para a organização e listagem dos livros em ordem alfabética, ela recebe os métodos de inserção recursiva, percorrer em ordem, remoção e sucessor, no método de inserção recursivo, ele compara os títulos dos livros. Se o título for alfabeticamente menor que o do nó atual, ele segue para a subárvore da esquerda caso contrário, para a da direita. No método `percorrer_em_ordem` visita os nós da esquerda, processa o nó atual e depois os da direita, garantindo que o relatório gerado apresente todos os livros em ordem alfabética. No método `_remover_recursivo` ele lida com a complexidade da remoção de nós na árvore. Se o nó a ser removido possui dois filhos, o sistema busca o seu sucessor (o valor mínimo da subárvore direita), substitui as informações do nó atual pelo sucessor e remove a duplicata original, mantendo a integridade da árvore.

2.6. Biblioteca

A classe responsável pelas funções nativas do sistema de biblioteca ela é responsável por manter as instâncias de `Lista_Encadeada`, `Arvore_Binaria`, `Tabela_Hash` e `Pilha`, garantindo que qualquer alteração no acervo reflita em todas as estruturas simultaneamente.

Os métodos principais são `cadastrar_livro(self, livro)`, `remover_livro(self, livro_isbn)`, `realizar_emprestimo(self, isbn, nome_usuario)`, `realizar_devolucao(self, isbn)`, `desfazer_ultimo_emprestimo(self)`, `gerar_relatorio_geral(self)`.

No método de `cadastrar_livro` a função realiza a validação inicial do cadastro. Ela verifica se o ISBN já existe na `Tabela_Hash` para evitar duplicatas. Se o ISBN for inédito, ela executa a inserção "tripla": adiciona o livro na `Tabela_Hash` (para busca $O(1)$), na `Lista_Encadeada` (para controle geral) e na `Arvore_Binaria` (para organização por título). Esse processo garante a consistência total do sistema.

No método `remover_livro(self, livro_isbn)`, a função busca o livro na `Tabela_Hash` para obter seu título e, a partir daí, remove o registro em todas as três estruturas. Isso é crítico para evitar "lixo" de dados no sistema após a exclusão de uma obra.

No método de `realizar_emprestimo(self, isbn, nome_usuario)`, contém a lógica de controle de estoque. Se o livro tem exemplares (`qtd_exemplares > 0`), ela decrementa o estoque e registra a transação na Pilha de histórico. Se o estoque estiver zerado, ela acessa o `Livro_Node` correspondente na árvore (utilizando `buscar_nodo_recursivo`) e enfileira o usuário na `fila_espera` específica daquele livro. Cada etapa gera um registro na Pilha, permitindo que operações de empréstimo ou entrada na fila sejam posteriormente desfeitas.

No método de `realizar_devolucao(self, isbn)` a função atua como um gerenciador de fila. Ao receber um ISBN, ela verifica se há usuários aguardando por esse livro na `fila_espera`. Se houver, ela "desenfileira" o próximo usuário, transferindo a responsabilidade do livro automaticamente para ele. Se não houver ninguém na fila, o exemplar retorna ao estoque. Essa função evita que livros fiquem parados na prateleira se houver demanda reprimida.

No método de `desfazer_ultimo_emprestimo(self)` a função "desempilha" o último registro da Pilha e, baseada no `tipo_op` (`EMPRESTIMO`, `DEVOLUCAO` ou `ENTROU_FILA`), executa a "operação de compensação". Por exemplo, se a última ação foi um empréstimo, ela cancela a saída do exemplar e ajusta o estoque ou remove o usuário da fila, revertendo exatamente o estado da biblioteca ao momento anterior à transação.

E por fim no método de `gerar_relatorio_geral(self)` a função utiliza o método de percorrimento da `Arvore_Binaria` (`percorrer_em_ordem`) para listar todo o acervo. Como a árvore mantém os livros organizados alfabeticamente, o relatório é gerado de forma ordenada de maneira nativa, sem necessidade de algoritmos de ordenação adicionais.

3. FRONTEND – OTECA

A interface gráfica do sistema, denominada OTECA, foi desenvolvida utilizando a biblioteca CustomTkinter, que permite a criação de uma interface moderna e intuitiva, seguindo princípios de design flat e responsividade. O front-end atua como uma camada de abstração sobre o back-end, onde cada tela (Cadastro, Circulação e Consulta) foi estruturada em frames modulares que facilitam a navegação e a organização visual das funcionalidades obrigatórias.

Do ponto de vista funcional, o front-end gerencia a comunicação com as estruturas de dados através de buffers de captura de saída (io.StringIO), permitindo que as mensagens de erro e sucesso processadas pela classe Biblioteca sejam exibidas de forma clara em caixas de diálogo (messagebox) ou diretamente na interface de consulta. Este módulo também integra uma funcionalidade extra de gerenciamento de capas de livros, utilizando uma Tabela_Hash_Capas própria para associar caminhos de arquivos de imagem aos respectivos ISBNs.

The screenshot displays the OTECA - Sistema de Biblioteca web interface. The browser window title is "OTECA - Sistema de Biblioteca". The interface features a sidebar on the left with a "NAVEGAÇÃO" (Navigation) section containing three items: "Cadastrar Livro" (highlighted with a dark background), "Empréstimos" (Loans), and "Consultar Acervo" (Consult Collection). The main content area has a header with the "OTECA" logo, which includes a stylized red book icon. Below the header, the title "Cadastro de Livros no Acervo" (Book Registration in the Collection) is displayed. The registration form is a white box with a light gray border, containing several input fields and a confirmation button. The form fields are arranged in two columns: ISBN (Código de Barras), Ano de Publicação, Título do Livro, and Quantidade de Exemplares in the top row; Autor Principal and Capa do Livro (.png) in the bottom row. The "Capa do Livro" field includes a placeholder text "Nenhuma imagem anexada" and a button labeled "Anexar Imagem". A large blue button at the bottom of the form is labeled "Confirmar Cadastro do Livro".

OTECA - Sistema de Biblioteca

OTECA

Cadastro de Livros no Acervo

NAVEGAÇÃO

- Cadastrar Livro
- Empréstimos
- Consultar Acervo

ISBN (Código de Barras):

Ano de Publicação:

Título do Livro:

Quantidade de Exemplares:

Autor Principal:

Capa do Livro (.png):

Nenhuma imagem anexada

Anexar Imagem

Confirmar Cadastro do Livro

